

EMPLOYEE ATTRITION PREDICTION

Machine Learning + Explainable AI(SHAP) + Tableau

Complete Project Guidebook and Revision Document

1. Project Overview

This project predicts whether an employee is likely to leave an organization using the IBM HR Employee Attrition dataset. The workflow covers data analysis, preprocessing, classification models, imbalance handling, SHAP explainability, CSV export, and Tableau visualization.

Item	Details
Project type	Supervised binary classification
Dataset	IBM HR Employee Attrition dataset
Original shape	1,470 rows × 35 columns
Target	Attrition
Target encoding	No → 0; Yes → 1
Models	Logistic Regression and Random Forest
Imbalance handling	class_weight='balanced'
Explainability	SHAP
Dashboard	Tableau
Notebook	01_Data_Exploration.ipynb
Tableau workbook	Employee_Attrition_Analysis.twbx

Phase 1 → Setup
Phase 2 → Dataset selection → Load Dataset → Understand Dataset → EDA
Phase 3 → Data Cleaning
Phase 4 → Preprocessing
Phase 5 → ML Models
Phase 6 → Model Evaluation
Phase 7 → Explainable AI / SHAP
Phase 8 → Export Results
Phase 9 → Tableau Dashboard
Phase 10 → Final Analysis & Report

2. Phase 1 — Setup

- Created the project workspace in VS Code.
- Used a Python virtual environment.
- Created and used the Jupyter notebook 01_Data_Exploration.ipynb.
- Prepared dataset and output folders.
- Installed/imported the required Python libraries.

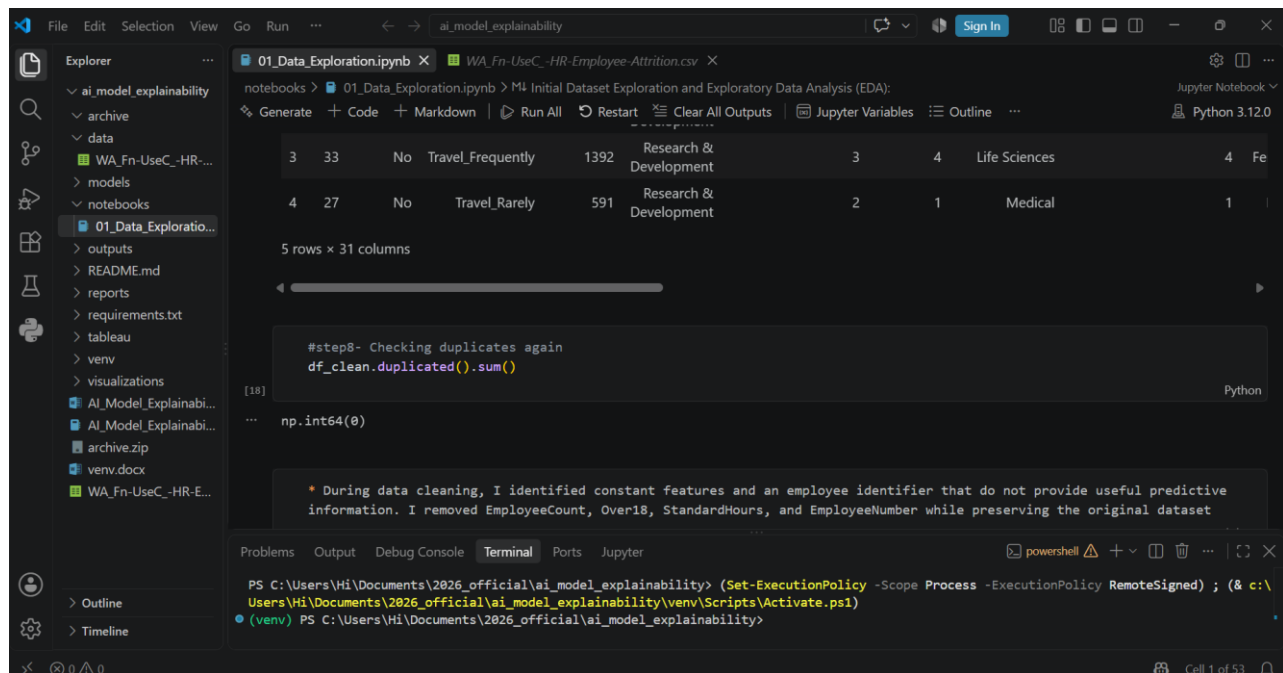
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

```

Key point: A virtual environment keeps project dependencies separate from other Python projects.



3. Phase 2 — Dataset Selection, Loading, Understanding, and EDA

3.1 Dataset selection

The IBM HR Employee Attrition dataset was selected because it contains employee-related features and a target column indicating whether an employee left.

3.2 Load dataset

```

df = pd.read_csv("../data/WA_Fn-UseC_-HR-Employee-Attrition.csv")
df.head()

```

3.3 Understand the dataset

```

df.shape
df.columns
df.info()
df.head()
df.sample(5)
df.nunique().sort_values()

```

- `df.shape` shows rows and columns.
- `df.columns` displays feature names.
- `df.info()` shows data types and non-null information.

- `df.head()` shows the first five rows.
- `df.sample(5)` shows random records.
- `df.nunique()` counts unique values in each column.

3.4 EDA

```
df["Attrition"].value_counts(normalize=True) * 100

sns.countplot(data=df, x="Attrition")
plt.show()

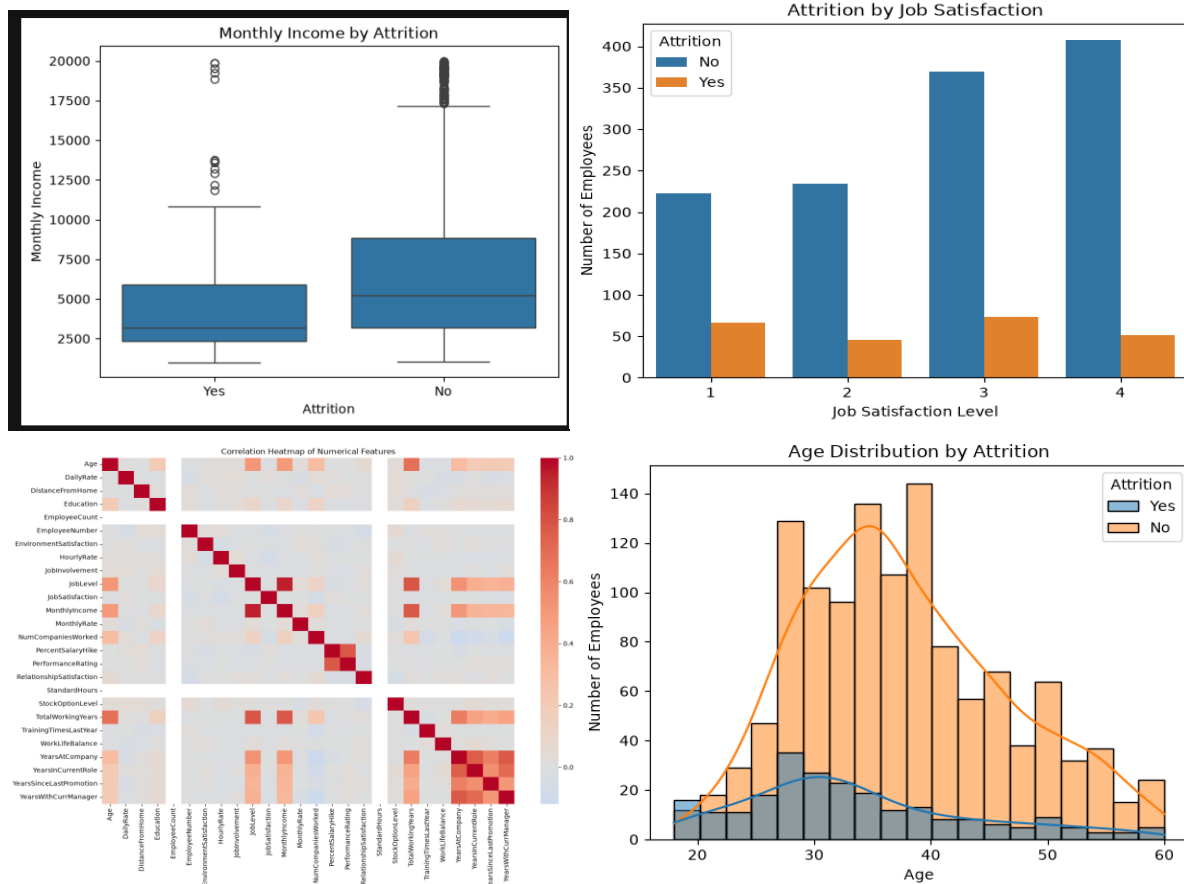
sns.countplot(data=df, x="OverTime", hue="Attrition")
plt.show()

sns.histplot(data=df, x="Age", hue="Attrition", kde=True)
plt.show()
```

The target counts were 1,233 employees with Attrition = 0 and 237 employees with Attrition = 1. Therefore, the target was imbalanced.

- `countplot` shows category counts.
- `hue` splits the plot into groups based on another variable.
- `histplot` shows the distribution of a numerical feature.
- `kde=True` adds a smooth density curve.

Phase result: The data was loaded, understood, and explored. Class imbalance and useful patterns such as OverTime versus Attrition were identified.



4. Phase 3 — Data Cleaning

```
df.duplicated().sum()
```

Duplicate records were checked.

Column removed	Reason
EmployeeCount	Constant value; no useful variation
Over18	Constant value; no useful variation
StandardHours	Constant value; no useful variation
EmployeeNumber	Identifier rather than a meaningful predictive feature

```
df = df.drop(columns=[  
    "EmployeeCount", "Over18", "StandardHours", "EmployeeNumber"  
)
```

Important: The original dataset was preserved before cleaning.

Phase result: Unhelpful constant columns and the employee identifier were removed.

5. Phase 4 — Preprocessing

5.1 Separate X and y

```
X = df.drop("Attrition", axis=1)  
y = df["Attrition"].map({"No": 0, "Yes": 1})
```

X contains input features. y contains the numeric target.

5.2 Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

- 80% training data and 20% testing data.
- random_state=42 makes the split reproducible.
- stratify=y preserves class proportions.
- Training shape: (1176, 34).
- Testing shape: (294, 34).

5.3 Identify feature types

```
categorical_features = X.select_dtypes(include=["object"]).columns  
numerical_features = X.select_dtypes(exclude=["object"]).columns
```

5.4 Build preprocessing

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("num", StandardScaler(), numerical_features),  
        ("cat", OneHotEncoder(  
            handle_unknown="ignore", drop="first"  
        ), categorical_features) ])
```

- StandardScaler standardizes numerical features.
- OneHotEncoder converts categories into binary columns.
- drop='first' reduces redundant dummy columns.
- handle_unknown='ignore' avoids errors for unseen categories.

Phase result: The data was split, encoded, scaled, and prepared for modeling.

6. Phase 5 — ML Models

6.1 Logistic Regression

```
logistic_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(
        max_iter=1000, random_state=42
    ))
])
logistic_model.fit(X_train, y_train)
y_pred_logistic = logistic_model.predict(X_test)
```

Logistic Regression estimates the probability of belonging to a class.

6.2 Random Forest

```
random_forest_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        n_estimators=100, random_state=42
    ))
])
random_forest_model.fit(X_train, y_train)
y_pred_forest = random_forest_model.predict(X_test)
```

Random Forest combines predictions from many decision trees.

6.3 Balanced models

```
logistic_balanced = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(
        max_iter=1000, class_weight="balanced",
        random_state=42
    ))
])

random_forest_balanced = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        n_estimators=100, class_weight="balanced",
        random_state=42
    ))
])

logistic_balanced.fit(X_train, y_train)
random_forest_balanced.fit(X_train, y_train)
```

Why class_weight='balanced'?: It gives more importance to the minority class during training.

Prediction-label correction: If predictions show No/Yes instead of 0/1, verify that y was created with `map({'No': 0, 'Yes': 1})` before splitting and training.

Phase result: Both standard and class-balanced versions of Logistic Regression and Random Forest were trained.

7. Phase 6 — Model Evaluation

Metric	Meaning
Accuracy	Percentage of all predictions that are correct
Precision	Of predicted attrition cases, how many actually left
Recall	Of actual attrition cases, how many were detected
F1-score	Balance between precision and recall
Confusion matrix	Counts true negatives, false positives, false negatives, and true positives

7.1 Original results

Model	Accuracy	Attrition Precision	Attrition Recall	Attrition F1
Logistic Regression	86.4%	64.0%	34.0%	44.4%
Random Forest	83.0%	36.4%	8.5%	13.8%

```
Logistic Regression
[[238  9]
 [ 31 16]]
```

```
Random Forest
[[240  7]
 [ 43  4]]
```

7.2 Balanced results

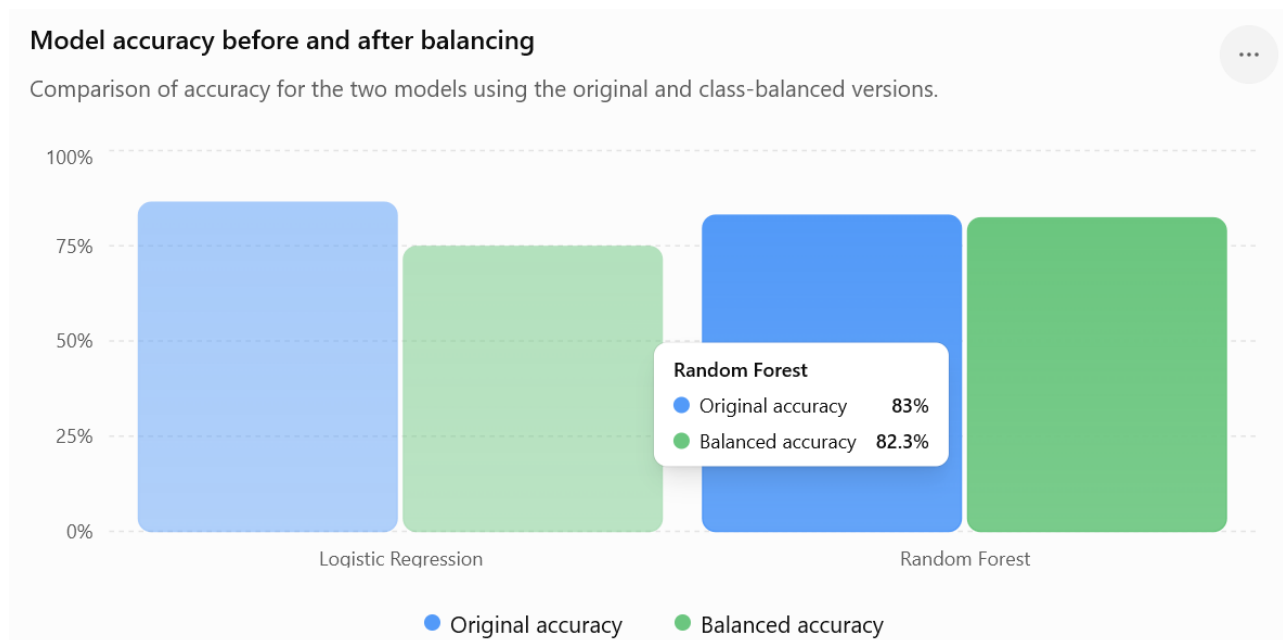
Model	Accuracy	Attrition Precision	Attrition Recall	Attrition F1
Balanced Logistic Regression	74.8%	34.8%	66.0%	45.6%
Balanced Random Forest	82.3%	41.4%	25.5%	31.6%

```
Balanced Logistic Regression
[[189 58]
 [ 16 31]]
```

```
Balanced Random Forest
[[230 17]
 [ 35 12]]
```

Class weighting improved minority-class recall, especially for Logistic Regression, while reducing its overall accuracy. This shows why accuracy alone is not enough for an imbalanced problem.

Phase result: The models were evaluated using accuracy, precision, recall, F1-score, and confusion matrices.



8. Phase 7 — Explainable AI / SHAP

```
!pip install shap
import shap
print("SHAP version:", shap.__version__)
```

SHAP version used: 0.52.0.

8.1 Prepare transformed test data

```
preprocessor_fitted = random_forest_balanced.named_steps["preprocessor"]
X_test_transformed = preprocessor_fitted.transform(X_test)
feature_names = preprocessor_fitted.get_feature_names_out()
```

The transformed test data had shape (294, 47).

8.2 Create explainer

```
forest_classifier = random_forest_balanced.named_steps["classifier"]
explainer = shap.TreeExplainer(forest_classifier)
shap_values = explainer.shap_values(X_test_transformed)
```

8.3 Summary plot

```
shap.summary_plot(
    shap_values[:, :, 1],
    X_test_transformed,
    feature_names=feature_names
)
```

- Features near the top have greater overall influence.
- Red generally indicates higher feature values; blue indicates lower values.
- Right/left position indicates contribution toward/away from Attrition = 1.
- SHAP explains model behavior, not causation.

8.4 Individual employee explanation

```
employee_index = 0
employee_data = X_test.iloc[[employee_index]]
employee_prediction = random_forest_balanced.predict(employee_data)[0]

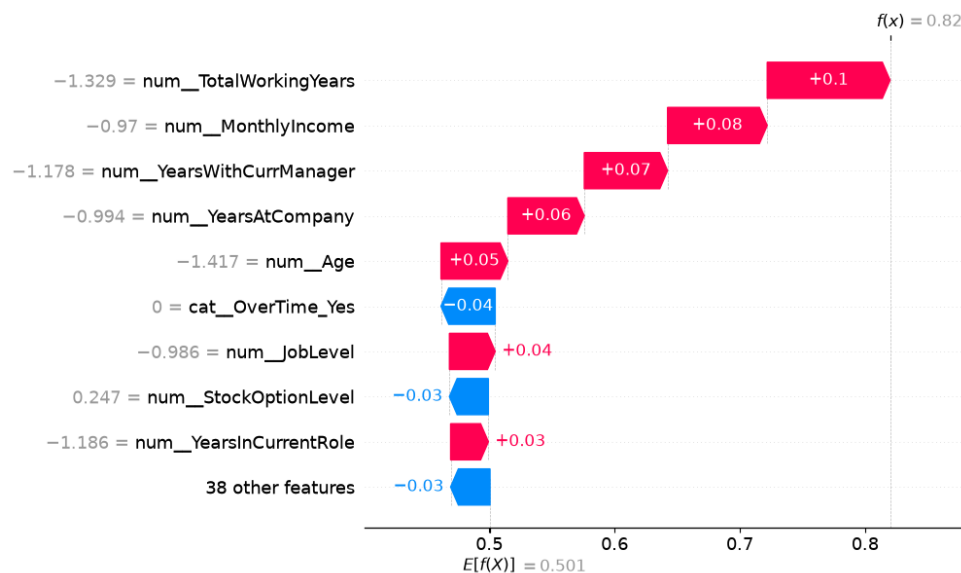
print("Actual Attrition:", y_test.iloc[employee_index])
print("Predicted Attrition:", employee_prediction)
```

For employee index 0, the actual value was 0 and the prediction was 1. This is a false positive.

```
employee_shap_values = shap_values[employee_index, :, 1]

shap.plots.waterfall(
    shap.Explanation(
        values=employee_shap_values,
        base_values=explainer.expected_value[1],
        data=X_test_transformed[employee_index],
        feature_names=feature_names
    )
)
```

Feature	SHAP value	Impact
num_TotalWorkingYears	+0.098093	Toward attrition
num_MonthlyIncome	+0.079570	Toward attrition
num_YearsWithCurrManager	+0.066486	Toward attrition
num_YearsAtCompany	+0.061016	Toward attrition
num_Age	+0.053386	Toward attrition
cat_OverTime_Yes	-0.043277	Away from attrition
num_JobLevel	+0.036558	Toward attrition
num_StockOptionLevel	-0.031118	Away from attrition
num_YearsInCurrentRole	+0.030148	Toward attrition
cat_JobRole_Sales Representative	+0.028068	Toward attrition

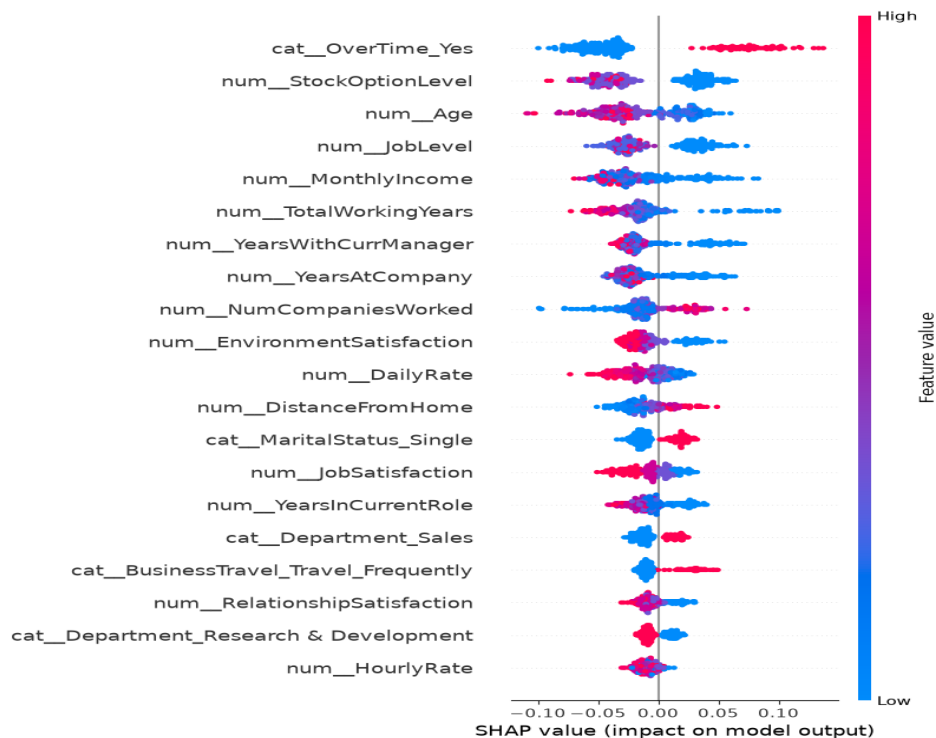


8.5 Global SHAP importance

```
global_importance = pd.DataFrame({
    "Feature": feature_names,
    "Mean SHAP Impact": abs(shap_values[:, :, 1]).mean(axis=0)
})
global_importance = global_importance.sort_values(
    "Mean SHAP Impact", ascending=False
)
global_importance.head(15)
```

Rank	Feature	Mean SHAP Impact
1	cat_OverTime_Yes	0.055349
2	num_StockOptionLevel	0.039397
3	num_Age	0.032104
4	num_JobLevel	0.029853
5	num_MonthlyIncome	0.029549
6	num_TotalWorkingYears	0.026511
7	num_YearsWithCurrManager	0.025710
8	num_YearsAtCompany	0.025169
9	num_NumCompaniesWorked	0.021845
10	num_EnvironmentSatisfaction	0.018375
11	num_DailyRate	0.018101
12	num_DistanceFromHome	0.017269
13	cat_MaritalStatus_Single	0.015606
14	num_JobSatisfaction	0.015055
15	num_YearsInCurrentRole	0.014997

Phase result: SHAP was used for global feature importance and individual prediction explanations.



9. Phase 8 — Export Results

9.1 Export predictions

```
prediction_probabilities = random_forest_balanced.predict_proba(X_test)

results_df = X_test.copy()
results_df["Actual_Attrition"] = y_test.values
results_df["Predicted_Attrition"] = y_pred_forest_balanced
results_df["Attrition_Probability"] = prediction_probabilities[:, 1]

results_df.to_csv(
    "../data/employee_attrition_predictions.csv",
    index=False
)
```

9.2 Export SHAP importance

```
global_importance.to_csv(
    "../data/shap_feature_importance.csv",
    index=False
)
```

9.3 Verification

```
Prediction file exists: True
SHAP file exists: True

Prediction file size: 46.65 KB
SHAP file size: 2.14 KB
```

10. Phase 9 — Tableau Dashboard

The prediction CSV was connected to Tableau as a Text File.

10.1 Worksheets created

1. Attrition Distribution
2. Overtime vs Attrition
3. Attrition by Job Role
4. Attrition Probability
5. Actual vs Predicted Attrition
6. Department vs Attrition

10.2 Filters added

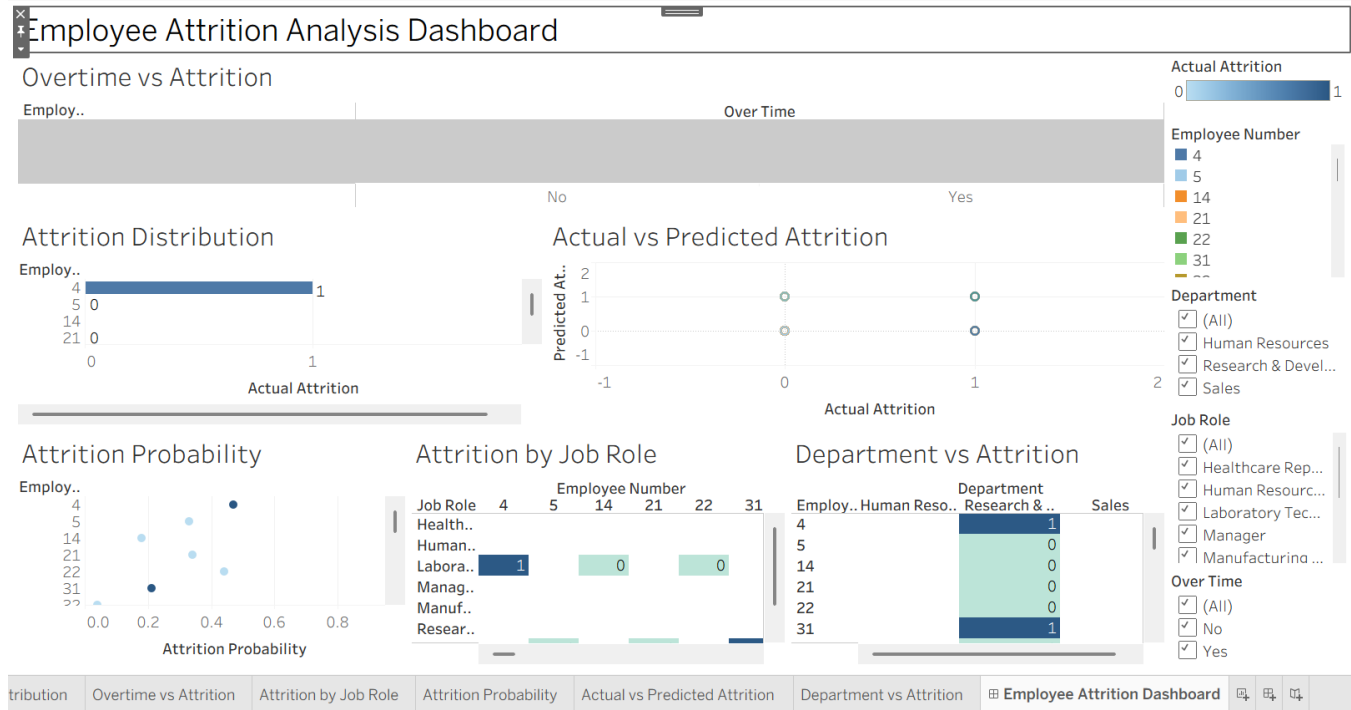
- Department
- JobRole
- OverTime

10.3 Dashboard workflow

7. Connect employee_attrition_predictions.csv.
8. Create each worksheet.
9. Place dimensions and measures into Rows, Columns, and Marks.
10. Add the worksheets to a dashboard.
11. Add Department, JobRole, and OverTime as filters.

12. Arrange and format the dashboard.
13. Save as Employee_Attrition_Analysis.twbx.

Phase result: A Tableau dashboard with six worksheets and three filters was created and saved.



11. Phase 10 — Final Analysis and Report

- The target was imbalanced, with fewer employees leaving than staying.
- Both Logistic Regression and Random Forest were trained.
- Class weighting improved minority-class recall, especially for Logistic Regression.
- Accuracy alone was not sufficient for judging the models.
- SHAP improved interpretability by showing global and individual contributions.
- Prediction and SHAP outputs were exported.
- Tableau presented the results through interactive charts and filters.

12. Quick Revision Sheet

Concept	Short meaning
Supervised learning	Learning from labeled input-output examples
Binary classification	Predicting one of two classes
Feature	Input variable used by a model
Target	Output variable to predict
EDA	Exploring distributions, patterns, and relationships
One-hot encoding	Converts categories into binary columns
Scaling	Puts numerical features on a comparable scale
Pipeline	Combines preprocessing and model steps
Class imbalance	One class has many more examples than another

Precision	Correct positive predictions among predicted positives
Recall	Detected positives among all actual positives
F1-score	Balance of precision and recall
SHAP	Explains feature contributions to predictions
False positive	Predicted attrition but employee stayed
False negative	Predicted stay but employee left
Tableau	Tool for interactive dashboards

13. Final Summary

The project developed an employee attrition prediction workflow using the IBM HR dataset. It included setup, data loading, EDA, cleaning, preprocessing, Logistic Regression and Random Forest models, imbalance handling, evaluation, SHAP explainability, CSV export, and Tableau visualization. The main learning was that accuracy alone can be misleading for imbalanced classification, so recall, precision, F1-score, confusion matrices, and explainability are important.